



《计算机科学导论》课程

数据结构与算法

上海交通大学计算机科学导论课程组



致谢：本课件由陈奕廷@CS2021D.SJTU编写；周昕逸@CS2021M.SJTU、宋昱龙@CS2021M.SJTU修改；高晓沅终校定稿

目 录

01

数据结构概述

02

典型数据结构

03

算法基本概念

02.典型数据结构：引言

你见过餐馆中一叠一叠的盘子吗？如果它们是按 $1, 2, \dots, n$ 的次序往上叠的，那么使用时候的次序应是什么样的？

必然是依从上往下的次序，即 $n, \dots, 2, 1$ 。它们遵循的是“**后进先出**”的规律，这正是“**栈**”的结构特点。

在日常生活中，为了维持正常的社会秩序而出现的常见现象是什么？

是“排队”。在计算机程序中，模拟排队的数据结构是“**队列**”，它们遵循的是“**先进先出**”的规律。

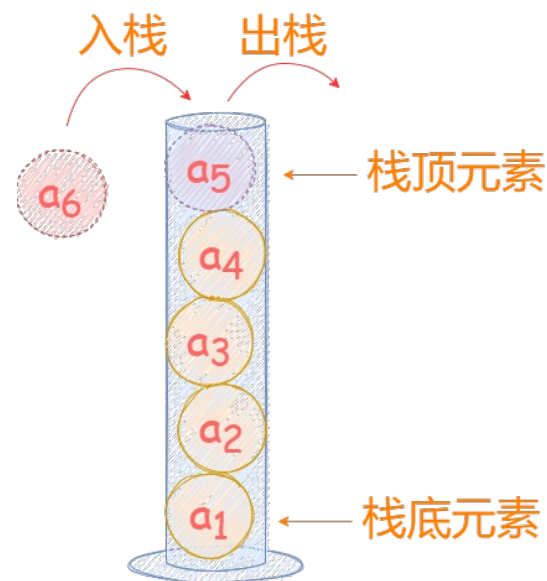
栈和队列也是线性表，不过是两种特殊的线性表。栈只允许在表的一端进行插入或删除操作，而队列只允许在表的一端进行插入操作、而在另一端进行删除操作。因而，栈和队列也可以被称作为操作受限的线性表。

02.典型数据结构：栈的定义

栈 (Stack) 是限定只能在表的一端进行插入和删除操作的线性表。

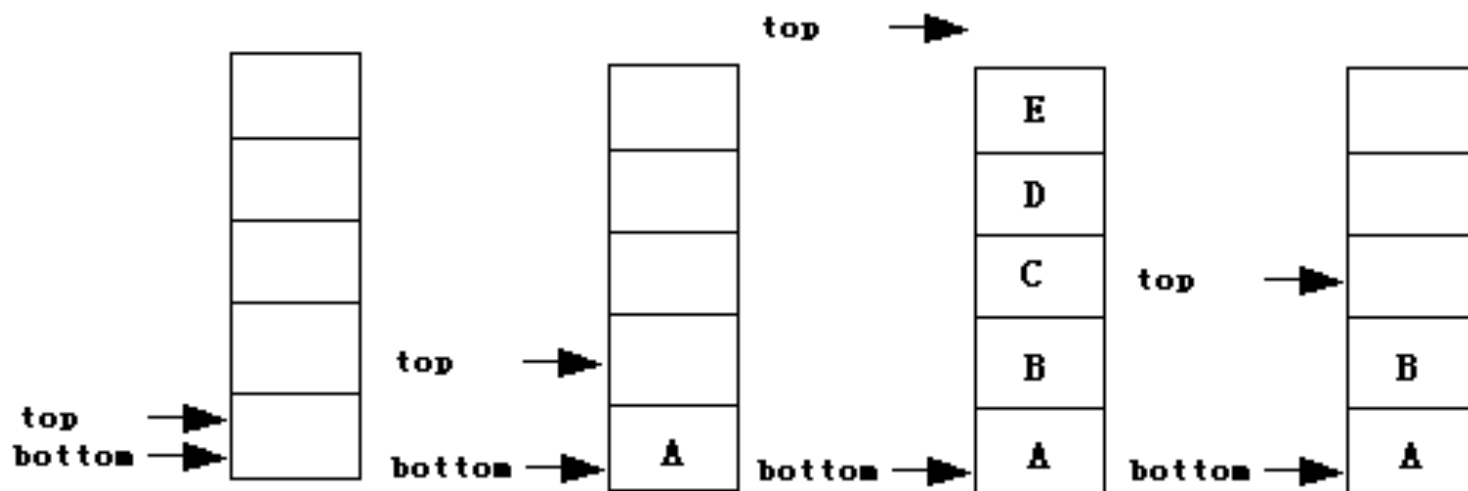
在表中，允许插入和删除的一端称作“**栈顶(top)**”，不允许插入和删除的另一端称作“**栈底(bottom)**”，当栈中无数据元素时，称为**空栈**。

往栈顶插入元素的操作为“**入栈**”或进栈(**push**)，称删除栈顶元素的操作为“**出栈**”或退栈(**pop**)。因为后入栈的元素先于先入栈的元素出栈，故被称为是一种“**后进先出**”的结构，因此又称**LIFO** (Last In First Out的缩写) 表。



02.典型数据结构：顺序栈示例

通常用一维数组表示栈的顺序存储结构。



栈顶指针和栈中元素之间的关系

注：

- 若bottom为null，则表明栈结构不存在。
- 若top=bottom表示空栈。
- 每插入新的栈顶元素时，指针top增1，删除栈顶元素时，top减1。因此，非空栈中的栈顶指针始终在栈顶元素的下一个位置上。

02.典型数据结构：栈的应用

例：算术表达式求值。

任何一个表达式都是由操作数 (operand)、运算符 (operator) 和界限符 (delimiter) 组成。

- 操作数可以是常数也可以是被说明为变量或常量的标识符。
- 运算符可以分为算术运算符、关系运算符和逻辑运算符等三类。
- 基本界限符有左右括弧和表达式结束符等。

为了叙述简洁，在此仅限于讨论只含二元运算符的算术表达式。

02.典型数据结构：栈的应用—算术表达式求值

最常用的算法称为“算符优先法”，任意两个相继出现的算符（运算符和界限符的统称）之间的优先关系只有三种： $\theta_1 > \theta_2$ ； $\theta_1 = \theta_2$ ； $\theta_1 < \theta_2$ 。

$\theta_1 \backslash \theta_2$	+	-	*	/	(	)	#
+	>	>	<	<	<	>	>
-	>	>	<	<	<	>	>
*	>	>	>	>	<	>	>
/	>	>	>	>	<	>	>
(	<	<	<	<	<	=	
)	>	>	>	>		>	>
#	<	<	<	<	<		=

算符间的优先关系

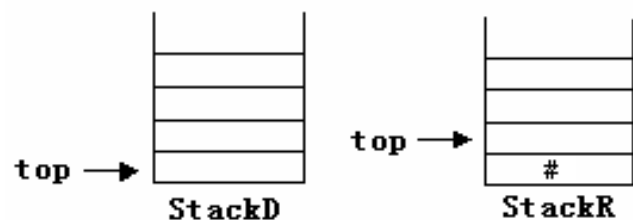
02.典型数据结构：计算步骤

为实现算符优先法，可以使用两个工作栈。一个称做StackR，用以寄存运算符。另一个称做StackD，用以寄存操作数或运算结果。

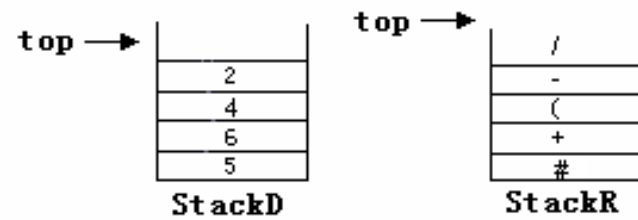
算法的基本思想：

- 首先将StackD置为空栈，表达式起始符“#”为运算符栈的栈底元素；
- 依次读入表达式中每个字符，若是操作数则进StackD栈；
- 若是运算符，则和StackR栈的栈顶运算符比较优先权（**栈顶运算符为 θ_1 ，当前读入运算符为 θ_2** ），若优先级高于栈顶元素则进栈（ $\theta_1 < \theta_2$ ），否则输出栈顶元素，从StackD中相应的输出两个操作数做相应运算，然后再与StackR中的栈顶元素进行优先级比较，
- 依次类推，直至整个表达式求值完毕（即StackR栈的栈顶元素和当前读入的字符均为“#”）

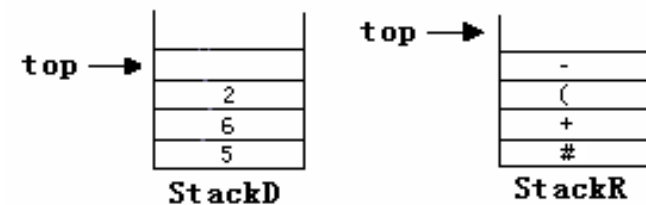
02.典型数据结构：计算 $5 + (6 - 4/2) * 3$ 过程



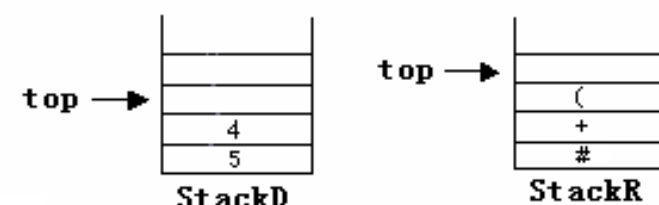
(a) 初始状态



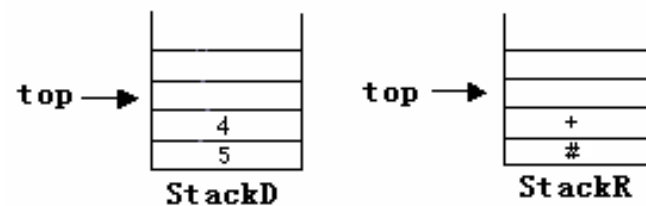
(b) 读出 5, +, (, 6, -, 4, /, 2



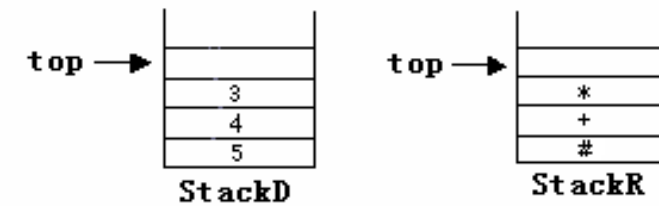
(c) 读出), 做运算 $T1 = 4/2 = 2$



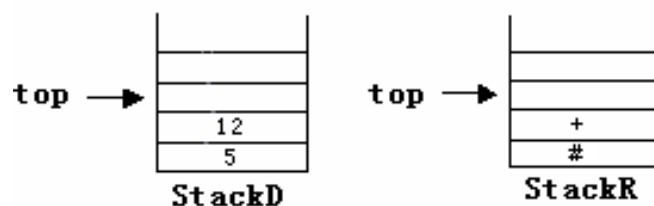
(d) 做运算 $T2 = 6 - 2 = 4$



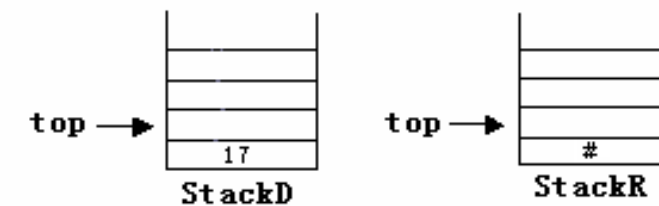
(e) 退 (



(f) 读出 *, 3



(g) 读出 \$, 做运算 $T3 = 4 * 3 = 12$



(h) 重新考虑 \$, 做运算 $T4 = 5 + 12 = 17$

02.典型数据结构：入栈过程

No	StackR	StackD	输入	主要操作	操作解释
0			<u>#</u> 5 + (6 - 4/2) * 3#	push(StackR, #)	起始操作符#入栈
1	#		<u>5</u> + (6 - 4/2) * 3#	push(StackD, 5)	操作数5入栈
2	#	5	<u>+</u> (6 - 4/2) * 3#	push(StackR, +)	栈顶“#” < “+”，入栈
3	# +	5	<u>(</u> 6 - 4/2) * 3#	push(StackR, (	“+” < “(”，入栈
4	# + (	5	<u>6</u> - 4/2) * 3#	push(StackD, 6)	操作数6入栈
5	# + (	5 6	<u>-</u> 4/2) * 3#	push(StackR, -)	“(” < “-”，入栈
6	# + (-	5 6	<u>4</u> /2) * 3#	push(StackD, 4)	操作数4入栈
7	# + (-	5 6 4	<u>/</u> 2) * 3#	push(StackR, /)	“-” < “/”，入栈
8	# + (-/	5 6 4	<u>2</u>) * 3#	push(StackD, 2)	操作数2入栈
9	# + (-/	5 6 4 2	<u>)</u> * 3#	operate(4/2)	“/” > “)”，执行4/2
10	# + (-	5 6 2	<u>-</u>) * 3#	operate(6 - 2)	“-” > “)”，执行6 - 2
11	# + (	5 4	<u>)</u> * 3#	pop(StackR)	消去一对括号
12	# +	5 4	<u>*</u> 3#	push(StackR, *)	“+” < “*”，入栈
13	# + *	5 4	<u>3</u> #	push(StackD, 3)	操作数3入栈
14	# + *	5 4 3	<u>#</u>	operate(4 * 3)	“*” > “#”，执行4 * 3
15	# +	5 12	<u>#</u>	operate(5 + 12)	“+” > “#”，执行5 + 12
16	#	17	<u>#</u>	pop(StackD)	读出StackD，返回结束

02.典型数据结构：栈的应用二—递归（Recursion）

在函数执行中，直接或间接调用自己的函数称为递归函数。常见递归方法有两种：

直接递归：函数直接调用其本身。

```
A( )  
{  
...  
  CALL A( )  
...  
}
```

间接递归：一个函数如果在调用其他函数时，又产生了对自身的调用。

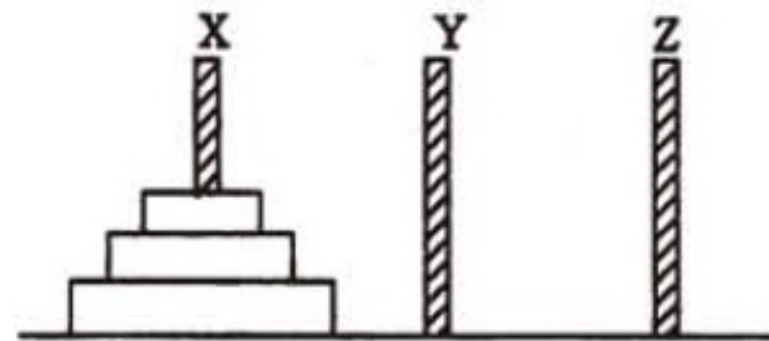
```
A( )          B ( )  
{             {  
...           ...  
  CALL B( )   CALL A( )  
...           ...  
}             }
```

02.典型数据结构：常见递归问题—汉诺塔(Hanoi)问题

假设有3个分别命名为X(From)、Y(Aux)和Z(to)的塔座，在塔座X上插有 n 个直径大小各不相同、依小到大编号为 $1, 2, \dots, n$ 的圆盘（如图所示）。

现要求将塔座X上的 n 个圆盘移至塔座Z上并仍按同样顺序叠排，圆盘移动时须遵循下列规则：

1. 每次只能移动一个圆盘；
2. 圆盘可以插在X、Y和Z中的任一塔座上；
3. 任何时刻都不能将一个较大的圆盘压在较小的圆盘之上



Algorithm 1: $\text{hanoi}(n, x, y, z)$

```
1 if  $n = 1$  then
2   |    $\text{move}(x, 1, z)$ ;
3 else
4   |    $\text{hanoi}(n - 1, x, z, y)$ ;
5   |    $\text{move}(x, n, z)$ ;
6   |    $\text{hanoi}(n - 1, y, x, z)$ ;
7 end
```

02.典型数据结构：汉诺塔(Hanoi)问题的调用栈

“栈”不是那三根柱子，尽管他们看起来也像“栈”。“递归算法中的栈”时，指的是程序运行时的调用栈（call stack），也就是计算机在执行递归函数时用来记录“还没完成的工作”的那块内存结构。

定义 Hanoi(N, X, Y, Z):
X = from (源)
Y = aux (辅助)
Z = to (目标)
规则:
 Hanoi(N-1, X, Z, Y)
 移动盘子 N: X -> Z
 Hanoi(N-1, Y, X, Z)
以下给出 N=3, 初始调用 Hanoi(3, X, Y, Z) 的调用栈变化。记号:
[push] 压栈
[pop] 弹栈
-> 实际搬盘动作
[push] Hanoi(3, X, Y, Z)
 调 1) [push] Hanoi(2, X, Z, Y)
 调 1) [push] Hanoi(1, X, Y, Z)
 调 1) [push] Hanoi(0, X, Z, Y)
 基例返回
 [pop] Hanoi(0, X, Z, Y)
 动作: 盘1 X -> Z
 调 3) [push] Hanoi(0, Y, X, Z)
 基例返回
 [pop] Hanoi(0, Y, X, Z)
 [pop] Hanoi(1, X, Y, Z)
 动作: 盘2 X -> Y

 调 3) [push] Hanoi(1, Z, X, Y)
 调 1) [push] Hanoi(0, Z, Y, X)
 基例返回
 [pop] Hanoi(0, Z, Y, X)
 动作: 盘1 Z -> Y
 调 3) [push] Hanoi(0, X, Z, Y)
 基例返回
 [pop] Hanoi(0, X, Z, Y)
 [pop] Hanoi(1, Z, X, Y)
 [pop] Hanoi(2, X, Z, Y)
 动作: 盘3 X -> Z
 调 3) [push] Hanoi(2, Y, X, Z)
 调 1) [push] Hanoi(1, Y, Z, X)
 调 1) [push] Hanoi(0, Y, X, Z)
 基例返回
 [pop] Hanoi(0, Y, X, Z)
 动作: 盘1 Y -> X
 调 3) [push] Hanoi(0, X, Y, Z)
 基例返回
 [pop] Hanoi(0, X, Y, Z)
 [pop] Hanoi(1, Y, Z, X)
 动作: 盘2 Y -> Z
 调 3) [push] Hanoi(1, X, Y, Z)
 调 1) [push] Hanoi(0, X, Z, Y)

 基例返回
 [pop] Hanoi(0, X, Z, Y)
 动作: 盘1 X -> Z
 调 3) [push] Hanoi(0, Y, X, Z)
 基例返回
 [pop] Hanoi(0, Y, X, Z)
 [pop] Hanoi(1, X, Y, Z)
 [pop] Hanoi(2, Y, X, Z)
 [pop] 最外层 Hanoi(3, X, Y, Z)
要点:
参数在两次递归调用中的重排分别是
(X, Z, Y) 和 (Y, X, Z)。
最大调用栈深度为 N（这里是 3）。
搬盘动作不会引起压栈或弹栈。

动画演示

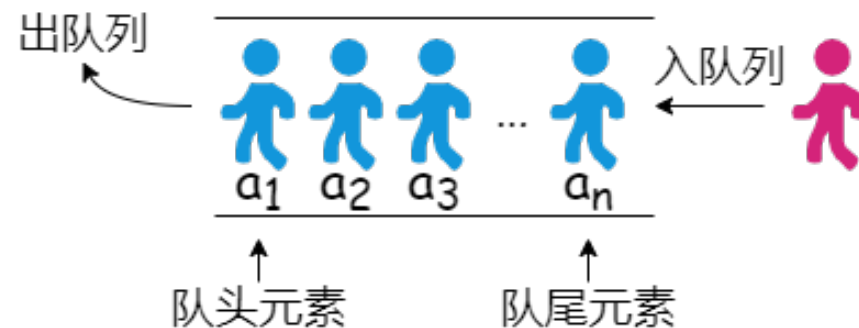
02.典型数据结构：队列

队列的定义：队列（Queue）是限定只能在表的一端进行插入和在另一端进行删除操作的线性表。在表中，允许插入的一端称作“**队尾(rear)**”，允许删除的另一端称作“**队头(front)**”。

不允许插队，新来的只能排在队尾，而最先进入队列的最早离开，故队列是一种“先进先出”的结构，因此又称**FIFO**（First In First Out的缩写）表。

当队列中无数据元素时，称为**空队列**。

队列的插入操作称为**进队或入队**，删除操作称为**出队**。



02.典型数据结构：队列的存储表示和操作的实现

队列的存储表示

- 顺序存储结构称为**顺序队列**，可延伸为**循环队列**

顺序队列

- 利用一组地址连续的存储单元依次存放队列中的数据元素。
- 一般情况下，我们使用一维数组来作为队列的顺序存储空间，另外再设立两个指示器：一个为指向队头元素位置的指示器front，另一个为指向队尾的元素位置的指示器rear。

顺序队列在程序设计中的一个典型应用例子是作业排队问题。

- 例如，在一个局域网上有一台共享的网络打印机，网上每个用户都可以将数据发送给网络打印机进行输出。为了保证不丢失数据，操作系统为网络的打印机生成一个"作业队列"，每个申请输出的"作业"应按先来后到的顺序排队，打印机从作业队列中逐个提取作业进行打印。
- 可利用循环队列结构解决两个设备速度不匹配的问题

02.典型数据结构：循环队列

问题背景：生产者-消费者速度差异

- 场景：**输入设备（生产者）快、输出设备（消费者）慢，或反之。
- 挑战：**直接传输导致数据丢失、阻塞、低效（无缓冲机制）。
- 示例：**硬盘读取快 vs. 显示器刷新慢。

循环队列核心原理

- 结构：**基于数组 + 模运算 ($\%N$)，队头(front)/队尾(rear)指针循环复用空间。
- 关键：**入/出队 $O(1)$ 时间；满： $(rear+1)\%N == front$ ；空： $rear == front$ （留空位区分）。
- 应用：**OS进程通信、网络Socket缓冲、嵌入式传感器。

动画演示

如何解决速度不匹配

- 缓冲作用：**生产者异步写入队列尾（快时填充）；消费者异步读出队列头（慢时等待）。
- 机制：**
 - 吸收多余数据（防溢出/丢失）。
 - 提供预存数据（减等待/阻塞）。
 - 循环设计：高效复用空间，最大化缓冲容量。

结果：异步解耦，实现稳定协调。

方面	线性队列	循环队列
空间利用	低（前端闲置）	高（全数组循环）
效率	易假溢出/阻塞	异步缓冲，少阻塞
适用	简单队列	设备I/O缓冲

02.典型数据结构：树(1)

树 (Tree) 是包含 n 个**结点(Node)** 的集合

层次结构顶部的结点为**根节点(Root)**

结点之间由**边(Edge)** 相连接

边定义了结点间的**父子关系**

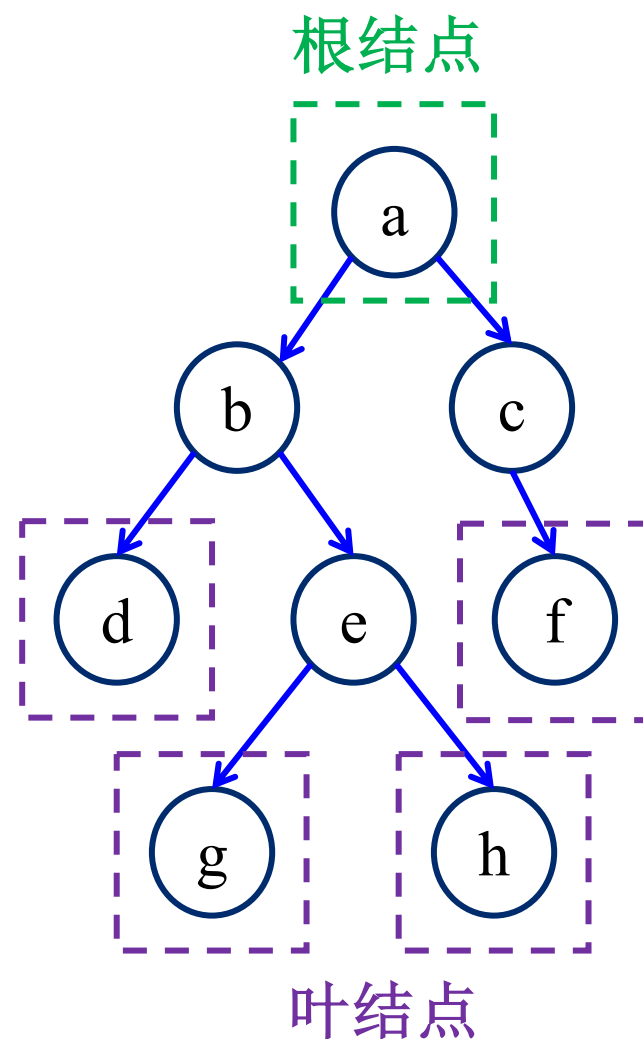
- 根节点无**父结点(Parent)**
- 其他结点只有一个**父结点**
- **子结点(Child)**数量称为结点的**度(Degree)**

无子结点的结点为**叶结点(Leaf)**

具有同一个父结点的结点互称**兄弟结点(Sibling)**

树的集合称为森林(Forest)

术语不重要，线性 $\rightarrow$ 非线性



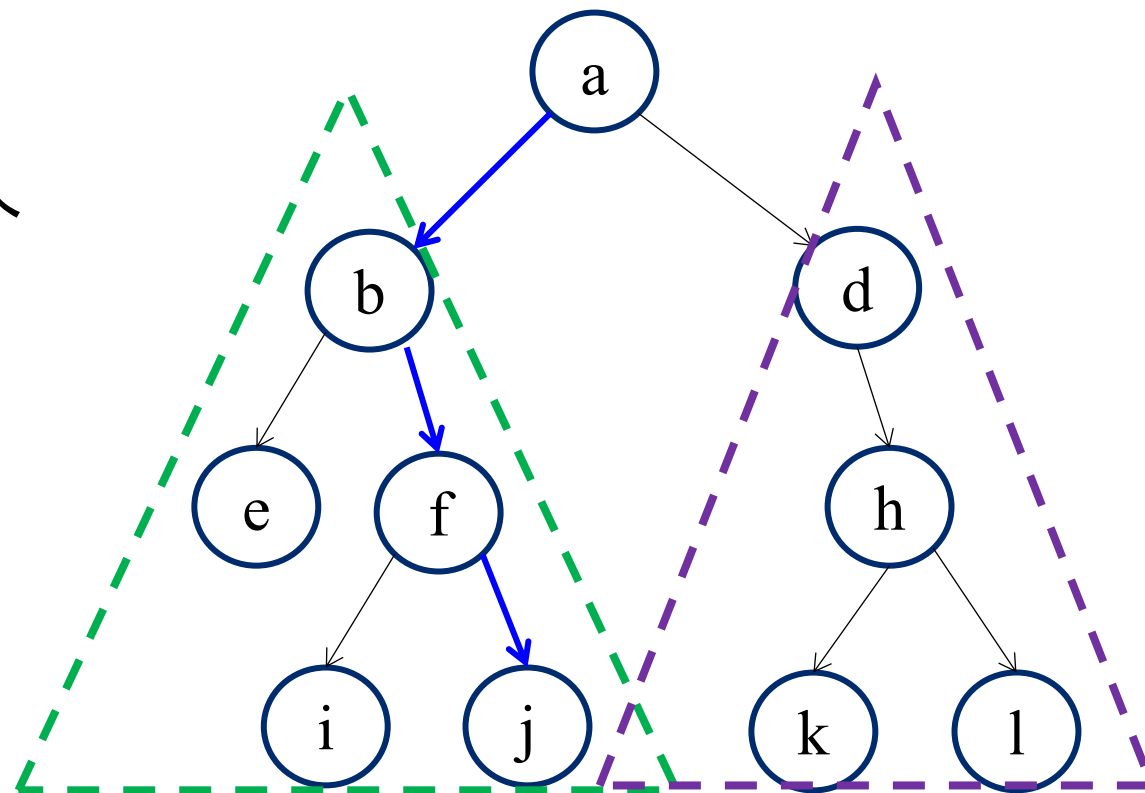
02.典型数据结构：树（2）

除根节点外的其余结点可以被分为多个不相交的集合，每个集合本身也是一棵树，称为原树的**子树**

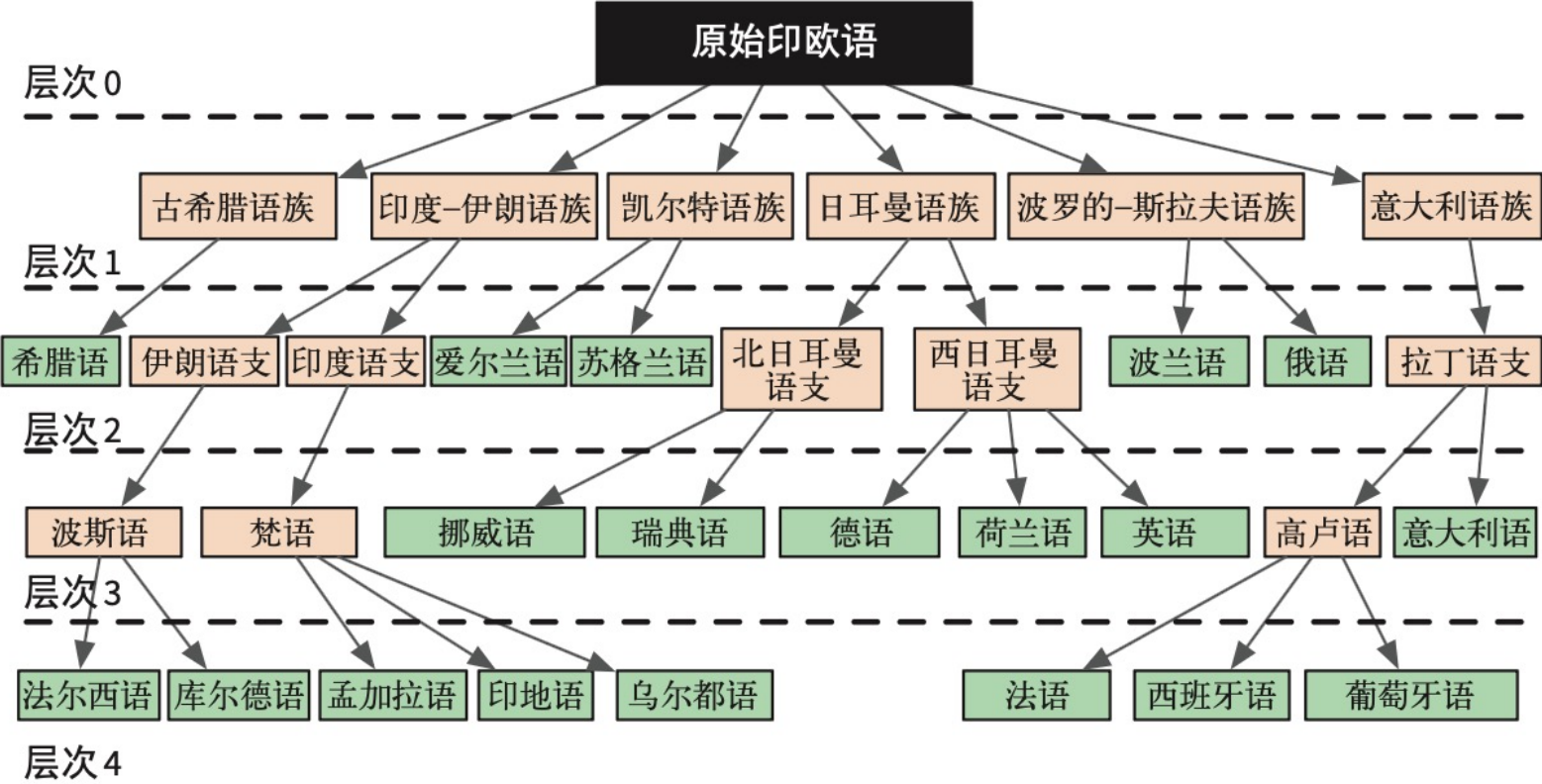
路径是按一定顺序排列的结点，其中下一个结点是上一个结点的子结点

- 例如：a->b->f->j是一条路径，长度为3
- 两结点间的路径是唯一的

若存在路径从结点a到结点b，则称a为b的**祖先**



02.典型数据结构：印欧语系起源树



树中的叶结点表示目前的语言（叶子无孩）

02.典型数据结构：二叉树（Binary Tree）

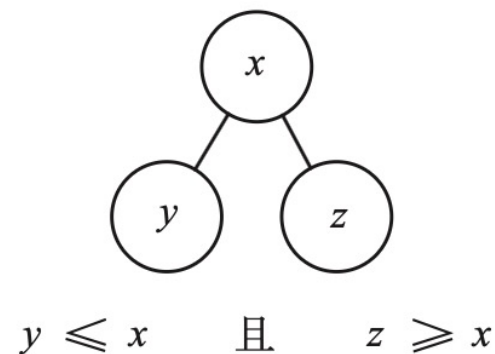
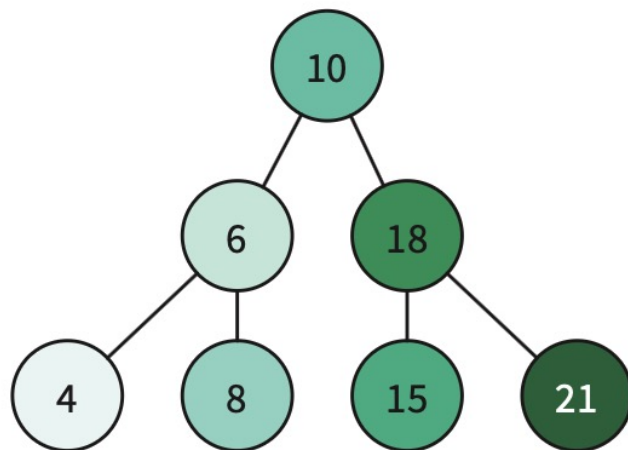
二叉查找树是一种能实现高效搜索的特殊树结构。

树中的结点最多可以有二个子结点，根据结点的键值对确定其位置。

父结点左侧的子结点必须小于父结点，而右侧的子结点必须大于父结点。

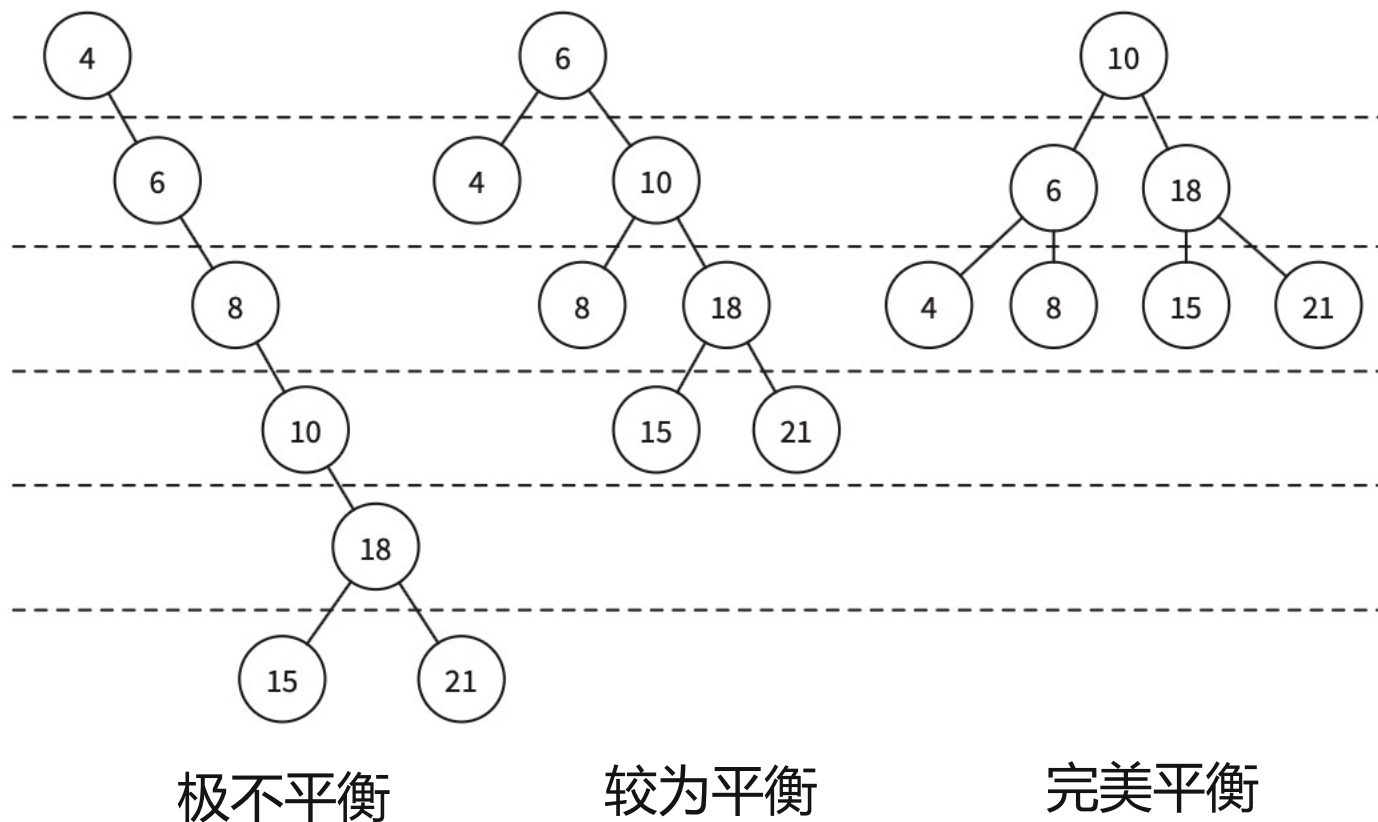
如果树遵循上述规则，就很容易在树中搜索具有给定键值对的某个结点。

插入元素时，对希望插入树中的值进行搜索。采用搜索中探索的最后一个结点，并使其右指针或左指针指向新结点。



02.典型数据结构：树的平衡

如果二叉查找树中插入的结点过多，最终将得到一棵极高的树，其中不少结点仅有一个子结点。例如，当插入结点的键值对总是大于前一个结点的键值对时，最后得到的数据结构将更接近于链表。



02.典型数据结构：优先队列（Priority Queue）

优先队列中每一个元素具有键值或优先级

- 优先队列可分为最大优先队列和最小优先队列，此处我们考虑最小优先队列

优先队列支持以下操作：

- isEmpty：判断优先队列是否为空
- Size：返回优先队列元素个数
- enqueue：在优先队列中加入一个新元素
- dequeueMin：去除优先级最小的元素
- getMin：得到优先级最小的元素

优先队列的应用：

- 银行：VIP顾客优先
- 医院：重病先治疗

网络带宽管理：

- 优先级较高的流量（如实时数据）一到达网络路由器便以最小的延迟转发

02.典型数据结构：优先队列的实现-二叉堆（Binary Heap）

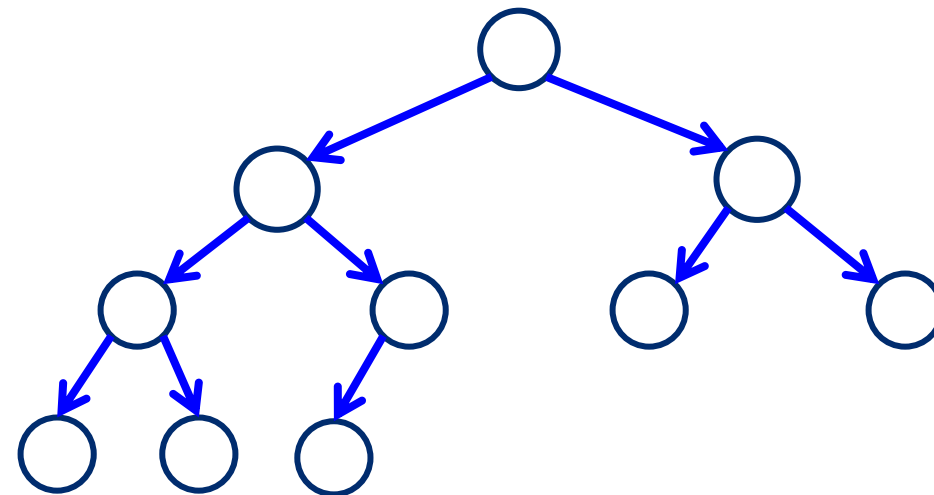
二叉堆是一棵完全二叉树

以最小二叉堆举例，任意结点的键值小于等于其子结点的键值

最小二叉堆的性质：对于任意子树，其根结点为键值最小的结点

二叉堆支持的操作：

- enqueue：加入一个新元素
- dequeueMin：去除优先级最小的元素
- getMin：得到优先级最小的元素



- 真二叉树（Proper Binary Tree）：所有节点都有0个或2个孩子
- 完全二叉树（Complete Binary Tree）：除了最底层其余层都满，且最底层从左往右顺序排列
- 完美二叉树（Perfect Binary Tree）：每层都满的二叉树，又称满二叉树

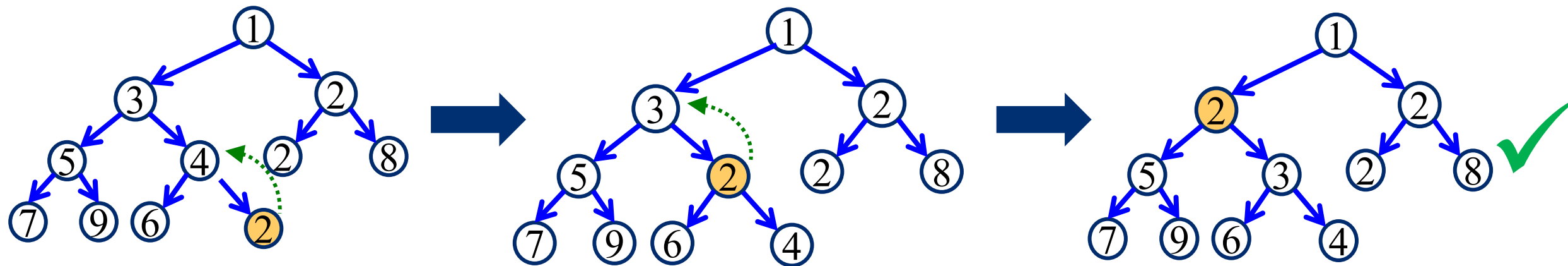
02.典型数据结构：二叉堆操作-enqueue

将新加入的元素插入为最右侧的叶结点

向上调整，将新加入的元素的键值与其父结点的键值比较，调整至合适的位置使二叉树满足二叉堆的性质

例：插入键值为2的元素：

动画演示

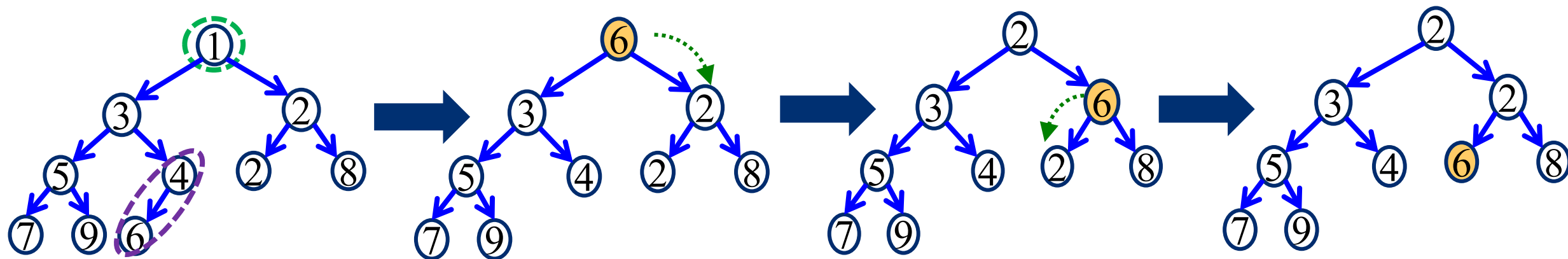


02.典型数据结构：二叉堆操作-dequeueMin

二叉堆的根结点为键值最小的结点，因此删除根结点，用最右侧的叶结点替代根结点

向下调整：将根结点键值与左、右子树的根结点键值相比较，与其中较小者交换；重复上述操作，直到其为叶结点，或键值小于等于左、右子树的根结点键值

例：删除下图二叉堆中键值最小的元素



目

录

01

数据结构概述

02

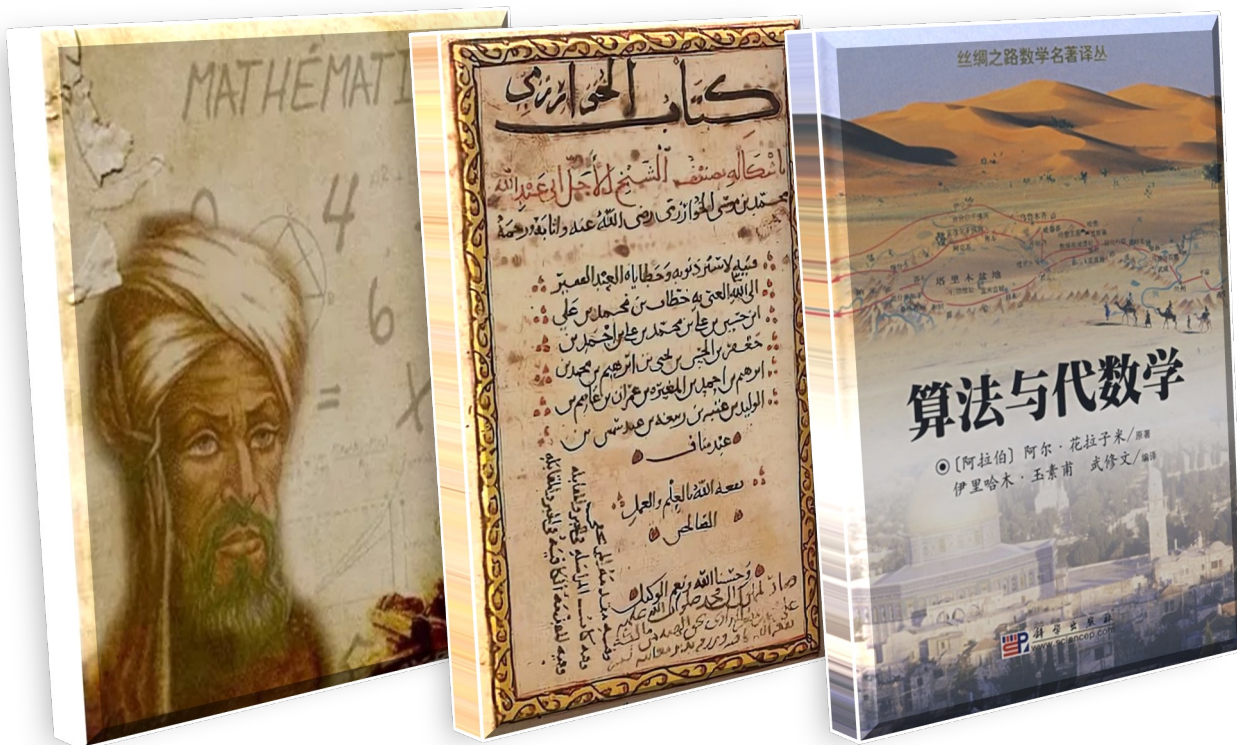
典型数据结构

03

算法基本概念

03.算法基本概念：算法的起源

算法的英文名称Algorithm，来源9世纪的波斯数学家花拉子密（al-Khwārizmī, 780A.D.-850A.D.），原为Algorism，意为演算法，出自Algoritmi，是花拉子密的拉丁文译名。



花拉子密雕像
(乌兹别克斯坦
花刺子模州)

- 超长的名字：穆罕默德·本·穆萨·阿尔·花拉子密 (Abu Abdulloh Muhammad ibn Muso al-Xorazmiy)
- 好好学习，也许以后你们的名字也会被命名变成一个概念~~😊😊

03.算法基本概念：为什么关心算法？

你每天都在“用到算法”

- 地图导航：从学校到火车站的最短/最快路线
- 电商/短视频：推荐与搜索，筛选“更可能喜欢”的内容
- 排队与调度：选课系统、医院叫号、打车分配



两个核心问题

- 能不能解决？（有办法吗）
- 多快能解决？（在给定资源下，多久出结果）

规模一变大，差别被放大

- 同样处理10万条数据：
 - ◆ 算法A: $O(n) \rightarrow 0.1$ 秒
 - ◆ 算法B: $O(n^2) \rightarrow 10,000$ 倍慢，约数分钟
- 直觉结论：硬件升级救得了一时，救不了规模爆炸



03.算法基本概念：算法的定义

算法的工程考量

- **成本**：时间=用户等待，钱=服务器资源
- **体验**：搜索/导航慢1秒，用户就流失
- **可持续**：数据每年翻倍，算法要“扛得住增长”

算法具有以下特征——《计算机程序设计艺术》

- **输入**：可能有0个或多个输入
- **输出**：至少产生一个结果
- **明确**：每一步都不含糊
- **有限**：一定會在有限步内结束
- **有效**：每步都能在当前机器上执行

思考：很多机器学习算法是基于概率和统计计算，是否满足这5性？

03.算法基本概念：示例（查找问题）

查找问题 (Searching Problem) :

给定一个包含 n 个数的数组 A ，在 A 中查找值为 x 的数并返回数组的索引值，否则返回-1

- 输入：包含 n 个数的数组 A
- 输出： x 在数组中的第一个位置或-1

线性查找法 (Linear Search) :

- 从数组第一个位置开始逐一查找
- 直到找到或查找到数组最后一位为止

二分查找 (Binary Search) :

- 排序数组
- 如何有效利用“有序性”？

Algorithm 1: LinearSearch($A[\cdot], x$)

Input : An array $A[1, \dots, n]$ of n elements,
an integer key x

Output: First index x in A , -1 if not found

```
1  $index \leftarrow -1$ ;  
2 for  $i \leftarrow 1$  to  $n$  do  
3   if  $A[i] = x$  then  
4      $index \leftarrow i$ ;  
5     break;  
6 return  $index$ ;
```

通过这个算法理解五性：输入、输出、明确、有限、有效

03.算法基本概念：计算复杂度

算法复杂度研究解决问题的难度或解决问题所需的资源数量

- 算法的效率影响算法的优劣
- 算法复杂度分析起源于1960年代，在1970至1980年代蓬勃发展

时间复杂度：解决问题所需的时间效率

空间复杂度：解决问题所需的存储空间效率

解决问题所需的资源受输入的大小，实现代码质量，硬件条件和算法复杂度决定。因此我们主要关注的是**解决问题所需资源随输入规模增大的速率**。

03.算法基本概念：O-Notation

O-Notation的定义

对于输入为自然数输出为非负实数的函数 $f(n)$ 和 $g(n)$ ，当满足

$$\exists c, \exists n_0. \forall n \geq n_0. f(n) \leq cg(n)$$

称 $f(n) = O(g(n))$

O-Notation表示了增长速率的上限。直观地说， $f(n)$ 随 n 的增长慢于某一常数倍的 $g(n)$

增长上界的直觉： $O(\dots)$ 表示最多像某函数那样增长 “屋顶”

03.算法基本概念：Ω-Notation

Ω-Notation的定义

对于输入为自然数输出为非负实数的函数 $f(n)$ 和 $g(n)$ ，当满足

$$\exists c, \exists n_0. \forall n \geq n_0. f(n) \geq cg(n)$$

称 $f(n) = \Omega(g(n))$

显然 $f(n) = O(g(n))$ 当且仅当 $g(n) = \Omega(f(n))$

增长下界的直觉：Ω (...) 表示至少像某函数那样增长 “地板”

03.算法基本概念： Θ -Notation

Θ -Notation的定义

对于输入为自然数输出为非负实数的函数 $f(n)$ 和 $g(n)$ ，当满足

$$f(n) = O(g(n)) \text{ 且 } f(n) = \Omega(g(n))$$

称 $f(n) = \Theta(g(n))$

显然 $f(n) = \Theta(g(n))$ 当且仅当 $g(n) = \Theta(f(n))$

直觉：当上界和下界一致时，我们说它的增长级别基本确定

03.算法基本概念：常见复杂度级别

$O(1)$ 常数 (哈希)

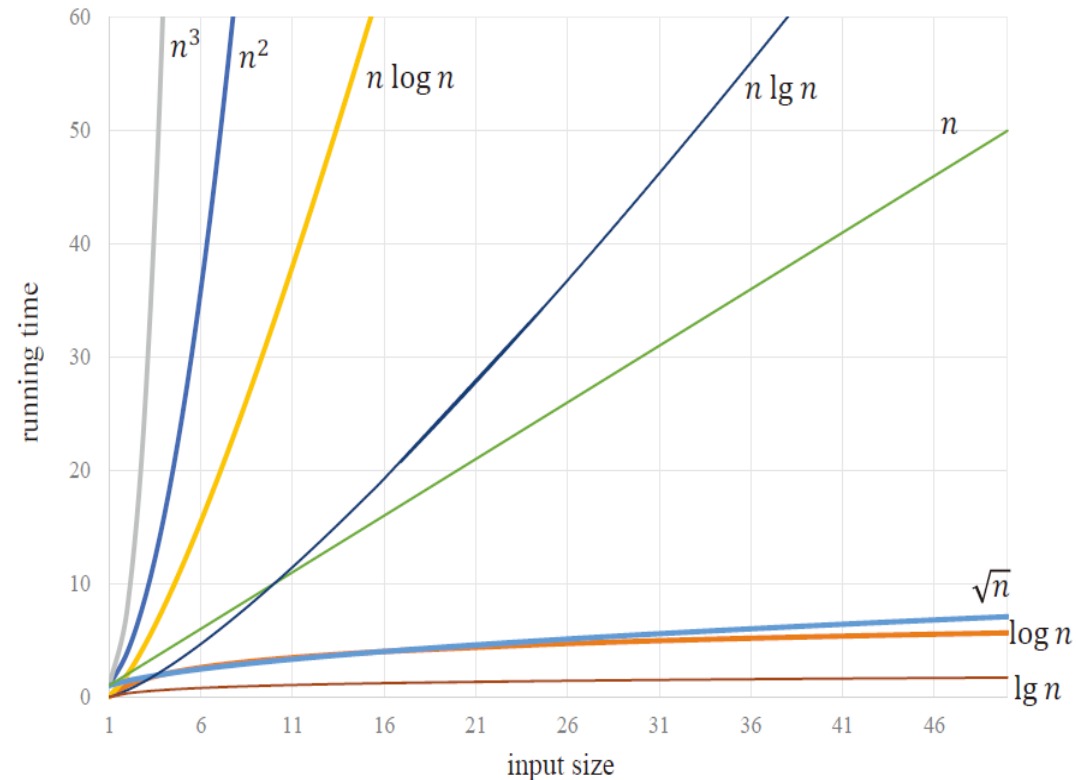
$O(\log n)$ 对数 (二分查找)

$O(n)$ 线性 (线性查找)

$O(n \log n)$ 高效排序 (快排/归并/堆排)

$O(n^2)$ 双重循环 (选择/冒泡最坏)

$O(2^n)$, $O(n!)$ 组合爆炸



只需记住快慢顺序，不必背定义

03.算法基本概念：示例（排序问题）

排序问题：给定一个包含 n 个数的数组 A ，对 A 内的数进行排序， A 中的数呈非降序

- 输入：包含 n 个数的数组 A
- 输出：排序后的数组

一种简单的排序算法-选择排序：
重复在未排序数字中找到最小的数，放在已排序数组的末尾。直到所有数组都已排序

Algorithm 8: SelectionSort($A[\cdot]$)

Input : An array $A[1, \dots, n]$ of n elements.

Output: $A[1, \dots, n]$ in nondecreasing order.

```
1 for  $i \leftarrow 1$  to  $n - 1$  do
2   for  $j \leftarrow i + 1$  to  $n$  do
3     if  $A[i] > A[j]$  then
4       swap  $A[i]$  and  $A[j]$ ;
5 return  $A[1, \dots, n]$ ;
```

对于任何一个输入数组，选择排序需要遍历整个数组，
总比较数 = $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$
因此时间复杂度为 $\Theta(n^2)$

数据结构概述

- 数据结构的概念、定义
- 线性表、顺序表

典型数据结构

- 栈
- 队列
- 树、二叉树、优先队列、二叉堆

算法基本概念

- 算法的起源、定义
- 算法复杂度

谢谢！

